

Open3D: A Modern Library for 3D Data Processing

Qian-Yi Zhou

Jaesik Park
Intel Labs

Vladlen Koltun

Abstract

Open3D is an open-source library that supports rapid development of software that deals with 3D data. The Open3D frontend exposes a set of carefully selected data structures and algorithms in both C++ and Python. The backend is highly optimized and is set up for parallelization. Open3D was developed from a clean slate with a small and carefully considered set of dependencies. It can be set up on different platforms and compiled from source with minimal effort. The code is clean, consistently styled, and maintained via a clear code review mechanism. Open3D has been used in a number of published research projects and is actively deployed in the cloud. We welcome contributions from the open-source community.

1. Introduction

The world is three-dimensional. Systems that operate in the physical world or deal with its simulation must often process three-dimensional data. Such data takes the form of point clouds, meshes, and other representations. Data in this form is produced by sensors such as LiDAR and depth cameras, and by software systems that support 3D reconstruction and modeling.

Despite the central role of 3D data in fields such as robotics and computer graphics, writing software that processes such data is quite laborious in comparison to other data types. For example, an image can be efficiently loaded and visualized with a few lines of OpenCV code [3]. A similarly easy to use software framework for 3D data has not emerged. A notable prior effort is the Point Cloud Library (PCL) [18]. Unfortunately, after an initial influx of open-source contributions, PCL became encumbered by bloat and is now largely dormant. Other open-source efforts include MeshLab [6], which provides a graphical user interface for processing meshes; libigl [11], which supports discrete differential geometry and related research; and a variety of solutions for image-based reconstruction [13]. Nevertheless, there is currently no open-source library that is fast, easy to use, supports common 3D data processing workflows, and is developed in accordance with modern software engineer-

ing practices.

Open3D was created to address this need. It is an open-source library that supports rapid development of software that deals with 3D data. The Open3D frontend exposes a set of carefully selected data structures and algorithms in both C++ and Python. The Open3D backend is implemented in C++11, is highly optimized, and is set up for OpenMP parallelization. Open3D was developed from a clean slate with a small and carefully considered set of dependencies. It can be set up on different platforms and compiled from source with minimal effort. The code is clean, consistently styled, and maintained via a clear code review mechanism.

Open3D has been in development since 2015 and has been used in a number of published research projects [22, 13, 15, 12]. It has been deployed and is currently running in the Tanks and Temples evaluation server [13].

Open3D is released open-source under the permissive MIT license and is available at <http://www.open3d.org>. We welcome contributions from the open-source community.

2. Design

Two primary design principles of Open3D are usefulness and ease of use [8]. Our key decisions can be traced to these two principles. Usefulness motivated support for popular representations, algorithms, and platforms. Ease of use served as a countervailing force that guarded against heavy-weight dependencies and feature creep.

Open3D provides data structures for three kinds of representations: point clouds, meshes, and RGB-D images. For each representation, we have implemented a complete set of basic processing algorithms such as I/O, sampling, visualization, and data conversion. In addition, we have implemented a collection of widely used algorithms, such as normal estimation [16], ICP registration [2, 4], and volumetric integration [7]. We have verified that the functionality of Open3D is sufficient by using it to implement complete workflows such as large-scale scene reconstruction [5, 15].

We carefully chose a small set of lightweight dependencies, including Eigen for linear algebra [9], GLFW for OpenGL window support, and FLANN for fast nearest neighbor search [14]. For easy compilation, powerful

Open3D: 用于 3D 数据处理的现代库

Qi-Yi Zhou Jaesik Park Vladlen Koltun 英特尔实验室

抽象的

Open3D is an open-source library that supports rapid development of software that deals with 3D data. The Open3D frontend exposes a set of carefully selected data structures and algorithms in both C++ and Python. The backend is highly optimized and is set up for parallelization. Open3D was developed from a clean slate with a small and carefully considered set of dependencies. It can be set up on different platforms and compiled from source with minimal effort. The code is clean, consistently styled, and maintained via a clear code review mechanism. Open3D has been used in a number of published research projects and is actively deployed in the cloud. We welcome contributions from the open-source community.

一、简介

世界是三维的。在物理世界中运行或处理其模拟的系统通常必须处理三维数据。此类数据采用点云、网格和其他表示形式。这种形式的数据由 LiDAR 和深度相机等传感器以及支持 3D 重建和建模的软件系统生成。

尽管 3D 数据在机器人和计算机图形学等领域发挥着核心作用，但与其他数据类型相比，编写处理此类数据的软件相当费力。例如，可以使用几行 OpenCV 代码有效地加载和可视化图像 [3]。同样易于使用的 3D 数据软件框架尚未出现。一个值得注意的先前工作是点云库 (PCL) [18]。不幸的是，在最初大量的开源贡献之后，PCL 变得臃肿不堪，现在基本上处于休眠状态。其他开源项目包括 MeshLab [6]，它提供了用于处理网格的图形用户界面；libigl [11]，支持离散微分几何及相关研究；以及各种基于图像的重建解决方案 [13]。然而，目前还没有一个快速、易于使用、支持常见 3D 数据处理工作流程、并且按照现代软件工程开发的开源库。

荷兰国际集团的做法。

Open3D 就是为了满足这一需求而创建的。它是一个开源库，支持快速开发处理 3D 数据的软件。Open3D 前端公开了一组精心挑选的 C++ 和 Python 数据结构和算法。Open3D 后端在 C++11 中实现，经过高度优化，并针对 OpenMP 并行化进行设置。Open3D 是从头开始开发的，具有一组经过仔细考虑的小型依赖项。它可以在不同的平台上设置，并以最小的努力从源代码编译。代码干净、风格一致，并通过清晰的代码审查机制进行维护。

Open3D 自 2015 年以来一直在开发，并已用于许多已发表的研究项目 [22,13,15,12]。它已经部署并目前正在 Tanks and Temples 评估服务器中运行 [13]。

Open3D 在 MIT 许可下开源发布，可从 <http://www.open3d.net> 获取。组织。我们欢迎开源社区的贡献。

2. 设计

Open3D 的两个主要设计原则是实用性和易用性 [8]。我们的关键决策可以追溯到这两个原则。实用性激发了对流行表示、算法和平台的支持。易用性是防止重量级依赖和功能蔓延的对抗力量。

Open3D 提供三种表示的数据结构：点云、网格和 RGB-D 图像。对于每种表示，我们都实现了一套完整的基本处理算法，例如 I/O、采样、可视化和数据转换。此外，我们还实现了一系列广泛使用的算法，例如法线估计 [16]、ICP 配准 [2, 4] 和体积积分 [7]。我们通过使用 Open3D 实现完整的工作流程（例如大规模场景重建）来验证 Open3D 的功能是否足够 [5, 15]。

我们仔细选择了一小组轻量级依赖项，包括用于线性代数的 Eigen [9]、用于 OpenGL 窗口支持的 GLFW 以及用于快速最近邻搜索的 FLANN [14]。方便编译，功能强大

but heavyweight libraries such as Boost and Ceres are excluded. Instead we use either lightweight alternatives (e.g., pybind11 instead of Boost.Python) or in-house implementations (e.g., for Gauss-Newton and Levenberg-Marquardt graph optimization). The source code of all dependencies is distributed as part of Open3D. The dependencies can thus be compiled from source if not automatically detected by the configuration script. This is particularly useful for compilation on operating systems that lack package management software, such as Microsoft Windows.

The development of Open3D started from a clean slate and the library is kept as simple as possible. Only algorithms that solve a problem of broad interest are added. If a problem has multiple solutions, we choose one that the community considers standard. A new algorithm is added only if its implementation demonstrates significantly stronger results on a well-known benchmark.

Open3D is written in standard C++11 and uses CMake to support common C++ toolchains including

- GCC 4.8 and later on Linux
- XCode 8.0 and later on OS X
- Visual Studio 2015 and later on Windows

A key feature of Open3D is ubiquitous Python binding. We follow the practice of successful computer vision and deep learning libraries [1, 3]: the backend is implemented in C++ and is exposed through frontend interfaces in Python. Developers use Python as a glue language to assemble components implemented in the backend.

Figure 1 shows code snippets for a simple 3D data processing workflow. An implementation using the Open3D Python interface is compared to an implementation using the Open3D C++ interface and to an implementation based on PCL [18]. The implementation using the Open3D Python interface is approximately half the length of the implementation using the Open3D C++ interface, and about five times shorter than the implementation based on PCL. As an added benefit, the Python code can be edited and debugged interactively in a Jupyter Notebook.

3. Functionality

Open3D has nine modules, listed in Table 1.

3.1. Data

The *Geometry* module implements three geometric representations: *PointCloud*, *TriangleMesh*, and *Image*.

The *PointCloud* data structure has three data fields: *PointCloud.points*, *PointCloud.normals*, *PointCloud.colors*. They are used to store coordinates, normals, and colors. The master data field is *PointCloud.points*. The other two fields are considered valid only when they have the same number of records as *PointCloud.points*. Open3D provides direct

memory access to these data fields via a numpy array. The following code sample demonstrates reading and accessing the point coordinates of a point cloud.

```
from py3d import *
import numpy as np
pointcloud = read_point_cloud('pointcloud.ply')
print(np.asarray(pointcloud.points))
```

Similarly, *TriangleMesh* has two master data fields – *TriangleMesh.vertices* and *TriangleMesh.triangles* – as well as three auxiliary data fields: *TriangleMesh.vertex_normals*, *TriangleMesh.vertex_colors*, and *TriangleMesh.triangle_normals*.

The *Image* data structure is implemented as a 2D or 3D array and can be directly converted to a numpy array. A pair of depth and color images of the same resolution can be combined into a data structure named *RGBDImage*. Since there is no standard depth image format, we have implemented depth image support for multiple datasets including NYU [19], TUM [20], SUN3D [21], and Redwood [5]. The following code sample reads a pair of RGB-D images from the TUM dataset and converts them to a point cloud.

```
from py3d import *
import numpy as np
depth = read_image('TUM_depth.png')
color = read_image('TUM_color.jpg')
rgbd = create_rgbd_image_from_tum_format(color,
    depth)
pointcloud = create_point_cloud_from_rgbd_image(
    rgbd, PinholeCameraIntrinsic(
        prime_sense_default))
```

3.2. Visualization

Open3D provides a function *draw_geometries()* for visualization. It takes a list of geometries as input, creates a window, and renders them simultaneously using OpenGL. We have implemented many functions in the visualizer, such as rotation, translation, and scaling via mouse operations, changing rendering style, and screen capture. A code sample using *draw_geometries()* and its result are shown in Figure 2.

In addition to *draw_geometries()*, Open3D has a number of sibling functions with more advanced functionality. *draw_geometries_with_custom_animation()* allows the programmer to define a custom view trajectory and play an animation in the GUI. *draw_geometries_with_animation_callback()* and *draw_geometries_with_key_callback()* accept Python callback functions as input. The callback function is called in an automatic animation loop, or upon a key press event. The following code sample shows a rotating point cloud using *draw_geometries_with_animation_callback()*.

但重量级库如 Boost 和 Ceres 被排除在外。相反，我们使用轻量级替代方案（例如，pybind11 而不是 Boost.Python）或内部实现（例如，用于 Gauss-Newton 和 Levenberg-Marquardt 图优化）。所有依赖项的源代码作为 Open3D 的一部分进行分发。因此，如果配置脚本没有自动检测到依赖项，则可以从源代码编译依赖项。这对于在缺少包管理软件的操作系统（例如 Microsoft Windows）上进行编译特别有用。

Open3D 的开发从头开始，并且库尽可能简单。仅添加解决广泛关注的问题的算法。如果一个问题有多种解决方案，我们会选择社区认为标准的一种。仅当新算法的实现在众所周知的基准上显示出明显更强的结果时，才会添加新算法。

Open3D 采用标准 C++11 编写，并使用 CMake 支持常见的 C++ 工具链，包括

- Linux 上的 GCC 4.8 及更高版本
- OS X 上的 XCode 8.0 及更高版本
- Windows 上的 Visual Studio 2015 及更高版本

Open3D 的一个关键特性是无处不在的 Python 绑定。我们遵循成功的计算机视觉和深度学习库的做法 [1, 3]：后端使用 C++ 实现，并通过 Python 中的前端接口公开。开发人员使用 Python 作为粘合语言来组装后端实现的组件。

图 1 显示了简单 3D 数据处理工作流程的代码片段。使用 Open3D Python 接口的实现与使用 Open3D C++ 接口的实现以及基于 PCL [18] 的实现进行了比较。使用 Open3D Python 接口的实现大约是使用 Open3D C++ 接口实现的长度的一半，比基于 PCL 的实现短大约五倍。作为一个额外的好处，Python 代码可以在 Jupyter Notebook 中进行交互编辑和调试。

3. 功能

Open3D 有九个模块，如表 1 所列。

3.1. 数据

Geometry 模块实现了三种几何表示：*PointCloud*、*TriangleMesh* 和 *Image*。

PointCloud 数据结构具有三个数据字段：*PointCloud.points*、*PointCloud.normals*、*PointCloud.colors*。它们用于存储坐标、法线和颜色。主数据字段是 *PointCloud.points*。其他两个字段仅当它们具有与 *PointCloud.points* 相同的记录数时才被视为有效。Open3D 提供直接

通过 *numpy* 数组对这些数据字段进行内存访问。以下代码示例演示了读取和访问点云的点坐标。

```
从 py3d 导入 *
import numpy as np
pointcloud = read_point_cloud('pointcloud.ply')
print(np.asarray(pointcloud.points))
```

同样，*TriangleMesh* 有两个主数据字段 – *TriangleMesh.vertices* 和 *TriangleMesh.triangles* – 以及三个辅助数据字段：*TriangleMesh.vertex_normals*、*TriangleMesh.vertex_colors* 和 *TriangleMesh.triangle_normals*。

Image 数据结构被实现为 2D 或 3D 数组，并且可以直接转换为 *numpy* 数组。一对相同分辨率的深度和彩色图像可以组合成一个名为 *RGBDImage* 的数据结构。由于没有标准的深度图像格式，我们为多个数据集实现了深度图像支持，包括 NYU [19]、TUM [20]、SUN3D [21] 和 Redwood [5]。以下代码示例从 TUM 数据集中读取一对 RGB-D 图像并将其转换为点云。

```
从 py3d 导入 *
导入 numpy 作为 np
深度 = read_image('TUM_深度.png')
颜色 = read_image('TUM_color.jpg')
rgbd = create_rgbd_image_from_tum_format(颜色, 深度)
点云 = create_point_cloud_from_rgbd_image(rgbd, PinholeCameraIntrinsic.prime_sense_default)
```

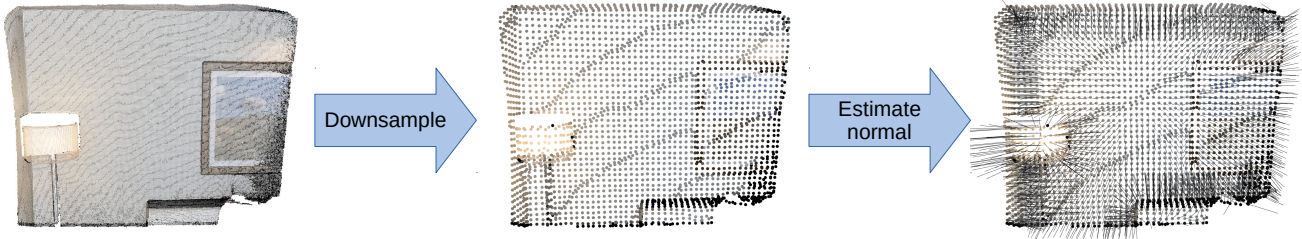
3.2. 可视化

Open3D 提供了一个用于可视化的函数 *draw_geometries()*。它以几何图形列表作为输入，创建一个窗口，并使用 OpenGL 同时渲染它们。我们在可视化工具中实现了许多功能，例如通过鼠标操作进行旋转、平移和缩放、更改渲染样式和屏幕捕获。使用 *draw_geometries()* 的代码示例及其结果如图 2 所示。

除了 *draw_geometries()* 之外，Open3D 还有许多具有更高级功能的兄弟函数。

draw_geometries_with_custom_animation() 允许程序员定义自定义视图轨迹并在 GUI 中播放动画。

draw_geometries_with_animation_callback() 和 *draw_geometries_with_key_callback()* 接受 Python 回调函数作为输入。回调函数在自动动画循环中或在按键事件时调用。以下代码示例显示了使用 *draw_geometries_with_animation_callback()* 的旋转点云。



(a) A simple 3D data processing task: load a point cloud, downsample it, and estimate normals.

```
from py3d import *
pointcloud = read_point_cloud('pointcloud.ply')
downsampled = voxel_down_sample(pointcloud, voxel_size = 0.05)
estimate_normals(downsampled, KDTreeSearchParamHybrid(radius = 0.1, max_nn = 30))
draw_geometries([downsampled])
```

(b) An implementation using the Open3D Python interface.

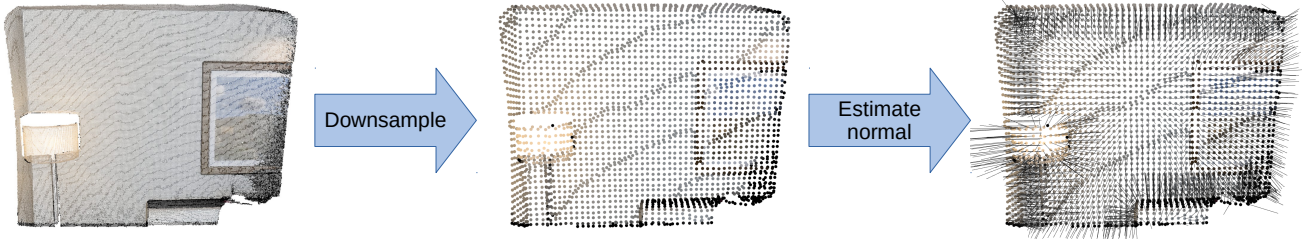
```
#include <Core/Core.h>
#include <IO/IO.h>
#include <Visualization/Visualization.h>
void main(int argc, char *argv[])
{
    using namespace three;
    auto pointcloud = CreatePointCloudFromFile(argv[1]);
    auto downsampled = VoxelDownSample(pointcloud, 0.05);
    EstimateNormals(*downsampled, KDTreeSearchParamHybrid(0.1, 30));
    DrawGeometry(downsampled);
}
```

(c) An implementation using the Open3D C++ interface.

```
#include <pcl/point_types.h>
#include <pcl/point_cloud.h>
#include <pcl/features/normal_3d.h>
#include <pcl/filters/voxel_grid.h>
#include <pcl/io/pcd_io.h>
#include <pcl/visualization/cloud_viewer.h>
void main(int argc, char *argv[])
{
    using namespace pcl;
    PointCloud<PointNormal>::Ptr pcd(new PointCloud<PointNormal>());
    io::loadPCDFile<PointNormal>(argv[1], *pcd);
    VoxelGrid<PointNormal> grid;
    grid.setLeafSize(0.05, 0.05, 0.05);
    grid.setInputCloud(pcd);
    grid.filter(*pcd);
    PointCloud<PointNormal>::Ptr result(new PointCloud<PointNormal>());
    NormalEstimation<PointNormal, PointNormal> est;
    est.setInputCloud(pcd);
    est.setRadiusSearch(0.1);
    est.compute(*result);
    pcl::visualization::PCLVisualizer viewer("PCL_Visualizer");
    viewer.addPointCloudNormals<PointNormal, PointNormal>(pcd, result, 1, 0.05);
    while (!viewer.wasStopped())
    {
        viewer.spinOnce();
    }
}
```

(d) An implementation based on PCL.

Figure 1: Code snippets for a simple 3D data processing workflow. (a) An illustration of the task. (b) An implementation using the Open3D Python interface. (b) An implementation using the Open3D C++ interface. (c) An implementation based on PCL [18]. The implementation using the Open3D Python interface is dramatically shorter and clearer than the implementation based on PCL.



(a) A simple 3D data processing task: load a point cloud, downsample it, and estimate normals.

```
from py3d import *
pointcloud = read_point_cloud('pointcloud.ply')
downsampled = voxel_down_sample(pointcloud, voxel_size = 0.05)
estimate_normals(downsampled, KDTreeSearchParamHybrid(radius = 0.1, max_nn = 30))
draw_geometries([downsampled])
```

(b) An implementation using the Open3D Python interface.

```
#include <Core/Core.h>
#include <IO/IO.h>
#include <Visualization/Visualization.h>
void main(int argc, char *argv[])
{
    using namespace three;
    auto pointcloud = CreatePointCloudFromFile(argv[1]);
    auto downsampled = VoxelDownSample(pointcloud, 0.05);
    EstimateNormals(*downsampled, KDTreeSearchParamHybrid(0.1, 30));
    DrawGeometry(downsampled);
}
```

(c) An implementation using the Open3D C++ interface.

```
#include <pcl/point_types.h>
#include <pcl/point_cloud.h>
#include <pcl/features/normal_3d.h>
#include <pcl/filters/voxel_grid.h>
#include <pcl/io/pcd_io.h>
#include <pcl/visualization/cloud_viewer.h>
void main(int argc, char *argv[])
{
    using namespace pcl;
    PointCloud<PointNormal>::Ptr pcd(new PointCloud<PointNormal>());
    io::loadPCDFile<PointNormal>(argv[1], *pcd);
    VoxelGrid<PointNormal> grid;
    grid.setLeafSize(0.05, 0.05, 0.05);
    grid.setInputCloud(pcd);
    grid.filter(*pcd);
    PointCloud<PointNormal>::Ptr result(new PointCloud<PointNormal>());
    NormalEstimation<PointNormal, PointNormal> est;
    est.setInputCloud(pcd);
    est.setRadiusSearch(0.1);
    est.compute(*result);
    pcl::visualization::PCLVisualizer viewer("PCL_Visualizer");
    viewer.addPointCloudNormals<PointNormal, PointNormal>(pcd, result, 1, 0.05);
    while (!viewer.wasStopped())
    {
        viewer.spinOnce();
    }
}
```

(d) An implementation based on PCL.

图 1: 简单 3D 数据处理工作流程的代码片段。(a) 任务说明。(b) 使用 Open3D Python 接口的实现。(c) 使用 Open3D C++ 接口的实现。(d) 基于 PCL 的实现[18]。使用 Open3D Python 接口的实现比基于 PCL 的实现要短得多、更清晰。

Module	Functionality
Geometry	Data structures and basic processing algorithms
Camera	Camera model and camera trajectory
Odometry	Tracking and alignment of RGB-D images
Registration	Global and local registration
Integration	Volumetric integration
I/O	Reading and writing 3D data files
Visualization	A customizable GUI for rendering 3D data with OpenGL
Utility	Helper functions such as console output, file system, and Eigen wrappers
Python	Open3D Python binding and tutorials

Table 1: Open3D modules.

```
from py3d import *
pointcloud = read_point_cloud('pointcloud.pcd')
mesh = read_triangle_mesh('mesh.ply')
mesh.compute_vertex_normals()
draw_geometries([pointcloud, mesh])
```

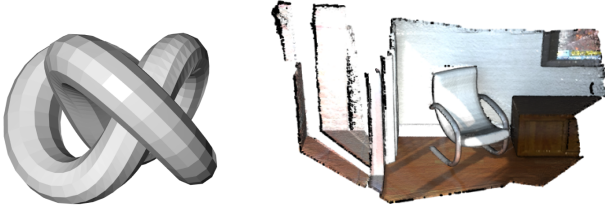


Figure 2: Visualize a mesh and a point cloud using `draw_geometries()`.

```
from py3d import *
pointcloud = read_point_cloud('pointcloud.pcd')
def rotate_view(vis):
    # Rotate the view by 10 degrees
    ctr = vis.get_view_control()
    ctr.rotate(10.0, 0.0)
    return False
draw_geometries_with_animation_callback([
    pointcloud], rotate_view)
```

In the backend of Open3D, these functions are implemented using the *Visualizer* class. This class is also exposed in the Python interface.

3.3. Registration

Open3D provides implementations of multiple state-of-the-art surface registration methods, including pairwise global registration, pairwise local refinement, and multiway registration using pose graph optimization. This section gives an example of a complete pairwise registration workflow for point clouds. The workflow begins by reading raw point clouds, downsampling them, and estimating normals:

```
from py3d import *
source = read_point_cloud('source.pcd')
target = read_point_cloud('target.pcd')
source_down = voxel_down_sample(source, 0.05)
target_down = voxel_down_sample(target, 0.05)
estimate_normals(source_down,
    KDTreeSearchParamHybrid(radius = 0.1, max_nn
    = 30))
estimate_normals(target_down,
    KDTreeSearchParamHybrid(radius = 0.1, max_nn
    = 30))
```

We then compute FPFH features and apply a RANSAC-based global registration algorithm [17]:

```
source_fpfh = compute_fpfh_feature(source_down,
    KDTreeSearchParamHybrid(radius = 0.25, max_nn
    = 100))
target_fpfh = compute_fpfh_feature(target_down,
    KDTreeSearchParamHybrid(radius = 0.25, max_nn
    = 100))
result_ransac =
    registration_ransac_based_on_feature_matching(
        source_down, target_down, source_fpfh,
        target_fpfh, max_correspondence_distance =
        0.075, TransformationEstimationPointToPoint(
        False), ransac_n = 4, [
        CorrespondenceCheckerBasedOnEdgeLength(0.9),
        CorrespondenceCheckerBasedOnDistance(0.075)],
        RANSACConvergenceCriteria(max_iteration =
        4000000, max_validation = 500))
```

We used profiling tools to analyze the RANSAC-based algorithm and found that its most time-consuming part is the validation of matching results. Thus, we give the user an option to specify a termination criterion via the *RANSAC-ConvergenceCriteria* parameter. In addition, we provide a set of functions that prune false matches early, including *CorrespondenceCheckerBasedOnEdgeLength* and *CorrespondenceCheckerBasedOnDistance*.

The final part of the pairwise registration workflow is ICP refinement [2, 4], applied to the original dense point clouds:

Module	Functionality
Geometry	Data structures and basic processing algorithms
Camera	Camera model and camera trajectory
Odometry	Tracking and alignment of RGB-D images
Registration	Global and local registration
Integration	Volumetric integration
I/O	Reading and writing 3D data files
Visualization	A customizable GUI for rendering 3D data with OpenGL
Utility	Helper functions such as console output, file system, and Eigen wrappers
Python	Open3D Python binding and tutorials

表 1: Open3D 模块。

```
from py3d import * pointcloud = read_point_cloud('pointcloud.pcd')
mesh = read_triangle_mesh('mesh.ply') mesh.compute_vertex_normals()
draw_geometries([pointcloud, mesh])
```

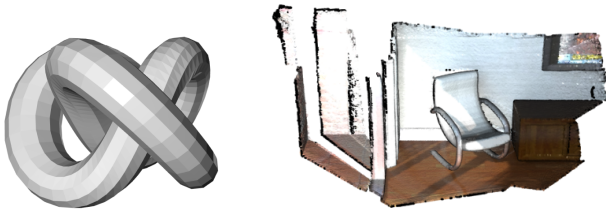


图 2: 使用 `draw_geometries()` 可视化网格和点云。

```
from py3d import * pointcloud = read_point_cloud('pointcloud.pcd')
def rotate_view(vis): # 将视图旋转 10 度
    ctr = vis.get_view_control()
    ctr.rotate(10.0, 0.0)
    return False
draw_geometries_with_animation_call_back([pointcloud], rotate_view)
```

在 Open3D 的后端，这些函数是使用 *Visualizer* 类实现的。这个类也在 Python 接口中公开。

3.3. 登记

Open3D 提供了多种最先进的表面配准方法的实现，包括成对全局配准、成对局部细化和使用位姿图优化的多路配准。本节给出了点云的完整成对配准工作流程的示例。工作流程首先读取原始点云，对其进行下采样并估计法线：

```
from py3d import * 源 = read_point_cloud('source.pcd') 目标 = read_point_cloud('target.pcd')
source_down = voxel_down_sample(source, 0.05)
target_down = voxel_down_sample(target, 0.05)
estimate_normals(source_down, KDTreeSearchParamHybrid(radius=0.1, max_nn=30))
estimate_normals(target_down, KDTreeSearchParamHybrid(radius=0.1, max_nn=30))
```

然后我们计算 FPFH 特征并应用基于 RANSAC 的全局配准算法 [17]：

```
source_fpfh = compute_fpfh_feature(source_down, KDTreeSearchParamHybrid(radius=0.25, max_nn=100))
target_fpfh = compute_fpfh_feature(target_down, KDTreeSearchParamHybrid(radius=0.25, max_nn=100))
result_ransac = Registration_ransac_based_on_feature_matching(source_down, target_down, source_fpfh, target_fpfh, max_correspondence_distance=0.075,
TransformationEstimationPointToPoint(False), ransac_n=4, [CorrespondenceCheckerBasedOnEdgeLength(0.9), CorrespondenceCheckerBasedOnDistance(0.075)], RANSACConvergenceCriteria(max_iteration=4000000, max_validation=500))
```

我们使用分析工具来分析基于 RANSAC 的算法，发现其最耗时的部分是匹配结果的验证。因此，我们为用户提供了通过 *RANSAC-ConvergenceCriteria* 参数指定终止标准的选项。此外，我们提供了一组尽早修剪错误匹配的函数，包括 *CorrespondenceCheckerBasedOnEdgeLength* 和 *CorrespondenceCheckerBasedOnDistance*。

成对配准工作流程的最后部分是 ICP 细化 [2, 4]，应用于原始密集点云：

```

result_icp = registration_icp(source, target,
    max_correspondence_distance = 0.02,
    result_ransac.transformation,
    TransformationEstimationPointToPlane())

```

Here *TransformationEstimationPointToPlane()* invokes a point-to-plane ICP algorithm. Other ICP variants are implemented as well. Intermediate and final results of the demonstrated registration procedure are shown in Figure 3.

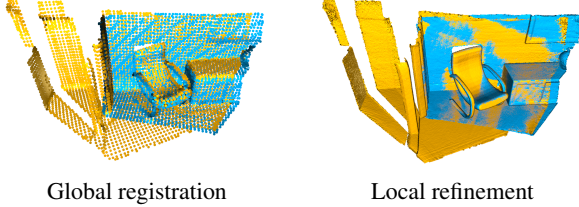


Figure 3: Intermediate and final results of pairwise registration.

3.4. Reconstruction

A sophisticated workflow that is demonstrated in an Open3D tutorial is a complete scene reconstruction system [5, 15]. The system is implemented as a Python script that uses many algorithms implemented in Open3D. It takes an RGB-D sequence as input and proceeds through three major steps.

1. Build local geometric surfaces $\{\mathbf{P}_i\}$ (referred to as fragments) from short subsequences of the input RGB-D sequence. There are three substeps: matching pairs of RGB-D images, robust pose graph optimization, and volumetric integration.
2. Globally align the fragments to obtain fragment poses $\{\mathbf{T}_i\}$ and a camera calibration function $\mathcal{C}(\cdot)$. There are four substeps: global registration between fragment pairs, robust pose graph optimization, ICP registration, and global non-rigid alignment.
3. Integrate RGB-D images to generate a mesh model for the scene.

Figure 4 shows a reconstruction produced by this pipeline for a scene from the SceneNN dataset [10]. The visualization was also done via Open3D.

4. Optimization

Open3D was engineered for high performance. We have optimized the C++ backend such that Open3D implementations are generally faster than their counterparts in other 3D processing libraries. For each major function, we used profiling tools to benchmark the execution of key steps. This

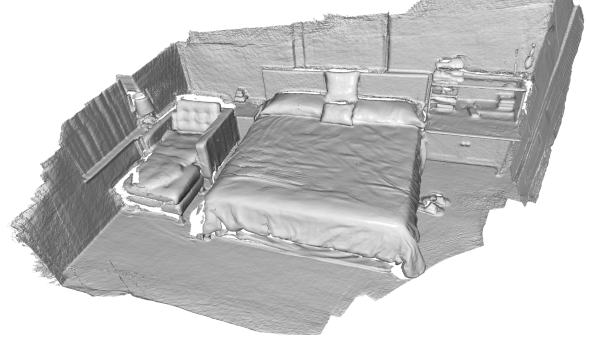


Figure 4: A scene reconstructed and visualized in Open3D.

analysis accelerated the running time of many functions by multiplicative factors. For example, our optimized implementation of the ICP algorithm is up to 25 times faster than its counterpart in PCL [18]. Our implementation of the reconstruction pipeline of Choi et al. [5] is up to an order of magnitude faster than the original implementation released by the authors.

A large number of functions are parallelized with OpenMP. Many functions, such as normal estimation, can be easily parallelized across data samples using “*#pragma omp for*” declarations. Another example of consequential parallelization can be found in our non-linear least-squares solvers. These functions optimize objectives of the form

$$L(\mathbf{x}) = \sum_i r_i^2(\mathbf{x}), \quad (1)$$

where $r_i(\mathbf{x})$ is a residual term. A step in a Gauss-Newton solver takes the current solution $\mathbf{x}^k$ and updates it to

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{J}_r^\top \mathbf{J}_r)^{-1} (\mathbf{J}_r^\top \mathbf{r}), \quad (2)$$

where $\mathbf{J}_r$ is the Jacobian matrix for the residual vector $\mathbf{r}$, both evaluated at $\mathbf{x}^k$. In the Open3D implementation, the most time-consuming part is the evaluation of $\mathbf{J}_r^\top \mathbf{J}_r$ and $\mathbf{J}_r^\top \mathbf{r}$. These were parallelized using OpenMP reduction. A lambda function specifies the computation of $\mathbf{J}_r^\top \mathbf{J}_r$ and $\mathbf{J}_r^\top \mathbf{r}$ for each data record. This lambda function is called in a reduction loop that sums over the matrices.

The parallelization of the Open3D backend accelerated the most time-consuming Open3D functions by a factor of 3-6 on a modern CPU.

5. Release

Open3D is released open-source under the permissive MIT license and is available at <http://www.open3d.org>. Ongoing development is coordinated via GitHub. Code changes are integrated via the following steps.

1. An issue is opened for a feature request or a bug fix.

```
result_icp = Registration_icp(源、目标、max_correspondence_distance = 0.02、result_ransac.transformation、TransformationEstimationPointToPlane())
```

这里 `TransformationEstimationPointToPlane()` 调用点到平面 ICP 算法。其他 ICP 变体也已实现。演示的注册过程的中间结果和最终结果如图 3 所示。

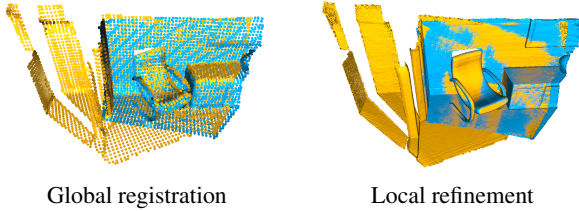


图 3: 成对配准的中间结果和最终结果。

3.4.重建

Open3D 教程中演示的复杂工作流程是一个完整的场景重建系统 [5, 15]。该系统以 Python 脚本的形式实现，该脚本使用许多在 Open3D 中实现的算法。它采用 RGB-D 序列作为输入并执行三个主要步骤。

1. 根据输入 RGB-D 序列的短子序列构建局部几何表面 $\{\mathbf{P}_i\}$ (称为片段)。共有三个子步骤：匹配 RGB-D 图像对、鲁棒位姿图优化和体积分。
2. 全局对齐片段以获得片段位姿 $\{\mathbf{T}_i\}$ 和相机校准函数 $\mathcal{C}(\cdot)$ 。有四个子步骤：片段对之间的全局配准、鲁棒位姿图优化、ICP 配准和全局非刚性对齐。
3. 集成 RGB-D 图像以生成场景的网格模型。

图 4 显示了该管道为 SceneNN 数据集 [10] 中的场景生成的重建结果。可视化也是通过 Open3D 完成的。

4. 优化

Open3D 专为高性能而设计。我们优化了 C++ 后端，使得 Open3D 实现通常比其他 3D 处理库中的对应版本更快。对于每个主要功能，我们使用分析工具来对关键步骤的执行进行基准测试。这

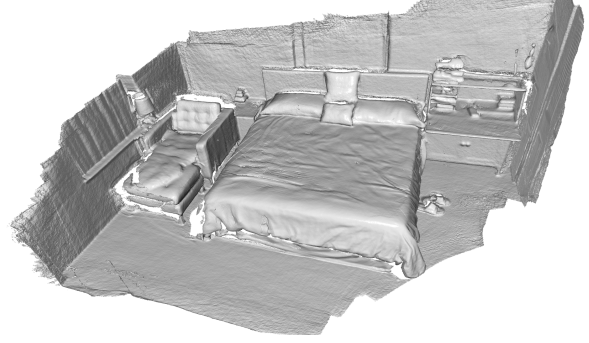


图 4: 在 Open3D 中重建和可视化的场景。

分析通过乘法因素加速了许多函数的运行时间。例如，我们对 ICP 算法的优化实现比 PCL 中的对应算法快 25 倍 [18]。我们实施了 Choi 等人的重建管道。[5] 比作者发布的原始实现快一个数量级。

大量功能与 OpenMP 并行。许多函数（例如正态估计）可以使用 “`#pragma omp for`” 声明轻松地跨数据样本并行化。后续并行化的另一个例子可以在我们的非线性最小二乘求解器中找到。这些功能优化了表单的目标

$$L(\mathbf{x}) = \sum_i r_i^2(\mathbf{x}), \quad (1)$$

其中 $r_i(\mathbf{x})$ 是残差项。高斯-牛顿求解器中的一个步骤采用当前解 $\mathbf{x}^k$ 并将其更新为

$$\mathbf{x}^{k+1} = \mathbf{x}^k - (\mathbf{J}_r^\top \mathbf{J}_r)^{-1} (\mathbf{J}_r^\top \mathbf{r}), \quad (2)$$

其中 $\mathbf{J}_r$ 是残差向量 $\mathbf{r}$ 的雅可比矩阵，两者均在 $\mathbf{x}^k$ 处计算。在 Open3D 实现中，最耗时的部分是对 $\mathbf{J}_r^\top \mathbf{J}_r$ 和 $\mathbf{J}_r^\top \mathbf{r}$ 的评估。这些使用 OpenMP 缩减进行并行化。lambda 函数指定每个数据记录的 $\mathbf{J}_r^\top \mathbf{J}_r$ 和 $\mathbf{J}_r^\top \mathbf{r}$ 的计算。该 lambda 函数在对矩阵求和的归约循环中调用。

Open3D 后端的并行化在现代 CPU 上将最耗时的 Open3D 函数加速了 3-6 倍。

5. 发布

Open3D 在 MIT 许可下开源发布，可从 <http://www.open3d.org> 获取。组织。持续的开发通过 GitHub 进行协调。代码更改通过以下步骤集成。

1. 针对功能请求或错误修复提出问题。

2. A developer starts a new branch on a personal fork of Open3D, writes code, and submits a pull request when ready.
3. The code change is reviewed and discussed in the pull request. Modifications are made to address issues raised in the discussion.
4. One of the admins merges the pull request to the master branch and closes the issue.

The code review process maintains a consistent coding style and adherence to modern C++ programming guidelines. Pull requests are automatically checked by a continuous integration service.

We hope that Open3D will be useful to a broad community of developers who deal with 3D data.

References

- [1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, et al. TensorFlow: A system for large-scale machine learning. In *OSDI*, 2016.
- [2] P. J. Besl and N. D. McKay. A method for registration of 3-D shapes. *PAMI*, 14(2), 1992.
- [3] G. Bradski. The OpenCV library. *Dr. Dobbs's Journal*, 25(11), 2000.
- [4] Y. Chen and G. G. Medioni. Object modelling by registration of multiple range images. *Image and Vision Computing*, 10(3), 1992.
- [5] S. Choi, Q.-Y. Zhou, and V. Koltun. Robust reconstruction of indoor scenes. In *CVPR*, 2015.
- [6] P. Cignoni, M. Callieri, M. Corsini, M. Dellepiane, F. Ganovelli, and G. Ranzuglia. MeshLab: An open-source mesh processing tool. In *Eurographics Italian Chapter Conference*, 2008.
- [7] B. Curless and M. Levoy. A volumetric method for building complex models from range images. In *SIGGRAPH*, 1996.
- [8] F. D. Davis. Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 1989.
- [9] G. Guennebaud, B. Jacob, et al. Eigen v3, 2010.
- [10] B. Hua, Q. Pham, D. T. Nguyen, M. Tran, L. Yu, and S. Yeung. SceneNN: A scene meshes dataset with annotations. In *3DV*, 2016.
- [11] A. Jacobson, D. Panozzo, et al. libigl: A simple C++ geometry processing library, 2016.
- [12] M. Khoury, Q.-Y. Zhou, and V. Koltun. Learning compact geometric features. In *ICCV*, 2017.
- [13] A. Knapitsch, J. Park, Q. Zhou, and V. Koltun. Tanks and temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4), 2017.
- [14] M. Muja and D. G. Lowe. Scalable nearest neighbor algorithms for high dimensional data. *PAMI*, 36(11), 2014.
- [15] J. Park, Q.-Y. Zhou, and V. Koltun. Colored point cloud registration revisited. In *ICCV*, 2017.
- [16] M. Pauly. Point primitives for interactive modeling and processing of 3D geometry. PhD thesis, ETH Zürich, 2003.
- [17] R. B. Rusu, N. Blodow, and M. Beetz. Fast point feature histograms (FPFH) for 3D registration. In *ICRA*, 2009.
- [18] R. B. Rusu and S. Cousins. 3D is here: Point Cloud Library (PCL). In *ICRA*, 2011.
- [19] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus. Indoor segmentation and support inference from RGBD images. In *ECCV*, 2012.
- [20] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IROS*, 2012.
- [21] J. Xiao, A. Owens, and A. Torralba. SUN3D: A database of big spaces reconstructed using SfM and object labels. In *ICCV*, 2013.
- [22] Q.-Y. Zhou, J. Park, and V. Koltun. Fast global registration. In *ECCV*, 2016.

2. 开发人员在 Open3D 的个人分支上启动一个新分支，编写代码，并在准备好后提交拉取请求。3. 代码更改在拉取请求中进行审查和讨论。针对讨论中提出的问题进行了修改。4. 一位管理员将拉取请求合并到主分支并关闭问题。

代码审查过程保持一致的编码风格并遵守现代 C++ 编程指南。持续集成服务会自动检查拉取请求。

我们希望 Open3D 对处理 3D 数据的广大开发人员社区有用。

参考

[1] M. Abadi、P. Barham、J. Chen、Z. Chen、A. Davis、J. Dean 等。TensorFlow：大规模机器学习系统。OSDI，2016 年。[2] P. J. Besl 和 N. D. McKay。一种 3D 形状配准方法。PAMI，14(2)，1992。[3] G. 布拉德斯基。OpenCV 库。Dr. Dobbs's Journal，25(11)，2000。[4] Y. Chen 和 G. G. Medioni。通过注册多个范围图像进行对象建模。Image and Vision Computing，10(3)，1992。[5] S. Choi，Q.-Y. 周和 V. 科尔通。室内场景的稳健重建。CVPR，2015 年。[6] P. Cignoni、M. Callieri、M. Corsini、M. Dellepiane、F. Ganovelli 和 G. Ranzuglia。MeshLab：开源网格处理工具。Eurographics Italian Chapter Conference，2008 年。[7] B. Curless 和 M. Levoy。一种从范围图像构建复杂模型的体积方法。SIG-GRAPH，1996 年。[8] F. D. Davis。感知有用性、感知易用性以及用户对信息技术的接受程度。MIS Quarterly，13(3)，1989。

[9] G. Guennebaud、B. Jacob 等人。Eigen v3，2010。[10] B. Hua、Q. Pham、D. T. Nguyen、M. Tran、L. Yu 和 S. Yeung。SceneNN：带有注释的场景网格数据集。3DV，2016 年。[11] A. Jacobson、D. Panozzo 等人。libigl：一个简单的 C++ 几何处理库，2016。[12] M. Khoury，Q.-Y. 周和 V. 科尔通。学习紧凑的几何特征。ICCV，2017 年。[13] A. Knapitsch、J. Park、Q. Zhou 和 V. Koltun。坦克和寺庙：大规模场景重建的基准。ACM Transactions on Graphics，36(4)，2017。[14] M. Muja 和 D. G. Lowe。适用于高维数据的可扩展最近邻算法。PAMI，36(11)，2014。[15] J. Park，Q.-Y. 周和 V. 科尔通。重新审视彩色点云配准。ICCV，2017 年。[16] M. Pauly。用于交互式建模和 3D 几何处理的点基元。博士论文，苏黎世联邦理工学院，2003 年。[17] R. B. Rusu、N. Blodow 和 M. Beetz。用于 3D 配准的快速点特征直方图 (FPFH)。ICRA，2009 年。[18] R. B. Rusu 和 S. Cousins。3D 在这里：点云库 (PCL)。ICRA，2011 年。[19] N. Silberman、D. Hoiem、P. Kohli 和 R. Fergus。室内分割并支持 RGBD 图像的推理。ECCV，2012 年。[20] J. Sturm、N. Engelhard、F. Endres、W. Burgard 和 D. Cremers。RGB-D SLAM 系统评估基准。IROS，2012 年。[21] J. Xiao、A. Owens 和 A. Torralba。SUN3D：使用 SfM 和对对象标签重建的大空间数据库。ICCV，2013 年。[22] Q.-Y. 周、J. Park 和 V. Koltun。快速全球注册。在 ECCV，2016 年。